

Laboratorio 01: Predicción de deterioro cognitivo con modelos de ensamble

Asignatura: *Deep Learning* (2026-01)

Profesor: Dr. Juan Bekios Calfa

1st Nabil Haddad Pomar

Escuela de Ingeniería

Universidad Católica del Norte

ICCI

nabil.haddad@alumnos.ucn.cl

2nd Tomás Méndez Díaz

Escuela de Ingeniería

Universidad Católica del Norte

ICCI

tomas.mendez@alumnos.ucn.cl

Resumen—Se entrenaron modelos de ensamble sobre un conjunto de datos con 15 atributos, utilizando un pipeline modular que incluye análisis de balance mediante entropía normalizada, preprocesamiento y optimización con `RandomizedSearchCV` y validación *Leave-One-Out*. Además, se implementó un clasificador *Naive Bayes* propio compatible con `scikit-learn`. Los resultados indican que `GDS_R2` logra el mejor balance de clases, mientras que `Bagging` y `Stacking` alcanzan los mejores desempeños (macro-AUC de 0.85 y 0.84). `Boosting` fue inestable, y el *Naive Bayes* propio mostró resultados competitivos.

I. INTRODUCCIÓN

El deterioro cognitivo es una condición progresiva que afecta la memoria y otras funciones mentales, representando un desafío relevante para el diagnóstico clínico temprano. En este contexto, el uso de pruebas psicométricas como el *Addenbrooke's Cognitive Examination III* (ACE-III), junto con técnicas de aprendizaje automático, ha mostrado ser una alternativa prometedora para apoyar la detección automatizada de demencia. [Fundación Pasqual Maragall, 2023]

El problema abordado en este laboratorio consiste en clasificar niveles de deterioro cognitivo a partir de un conjunto estructurado de 15 atributos predictivos derivados de este tipo de evaluaciones. El dataset original contiene múltiples codificaciones de la variable objetivo (GDS), las cuales no presentan el mismo comportamiento estadístico ni el mismo balance entre clases. Esta diferencia es crítica en clasificación multiclase, ya que distribuciones desbalanceadas pueden sesgar el modelo hacia la clase mayoritaria y degradar el desempeño en clases minoritarias.

En este contexto, el trabajo se enfocó en construir un pipeline reproducible capaz de comparar preprocesamientos, entrenar clasificadores base y reutilizar sus mejores configuraciones para formar modelos de ensamble. La lógica central consiste en realizar una búsqueda sistemática de hiperparámetros, persistir los mejores modelos y posteriormente ensamblarlos

mediante estrategias como `Stacking`, `Bagging` y `Boosting`.

Otro eje central del laboratorio fue el tratamiento del desbalance, abordado mediante decisiones complementarias: selección de una codificación objetivo más equilibrada utilizando entropía normalizada, partición estratificada de los datos, uso de métricas macro-promediadas, ponderación de clases en SVM y estimación de probabilidades a priori en el *Naive Bayes* propio a través del parámetro `fit_prior`.

Objetivos principales del trabajo:

- Analizar el balance de las distintas codificaciones objetivo y seleccionar una representación adecuada para entrenamiento multiclase.
- Comparar el impacto de dos estrategias de preprocesamiento: filtrado por correlación de Pearson y reducción de dimensionalidad con PCA.
- Entrenar clasificadores base con búsqueda aleatoria de hiperparámetros y validación cruzada *Leave-One-Out*.
- Construir y evaluar modelos de ensamble basados en los mejores estimadores encontrados.
- Implementar y validar un modelo *Naive Bayes* propio integrado al flujo general del proyecto.

II. MARCO TEÓRICO

El trabajo se sustenta en conceptos de estadística, validación experimental y aprendizaje supervisado. En primer lugar, la **entropía normalizada de Shannon** se utilizó para cuantificar cuán balanceada es cada codificación de la variable objetivo. Valores cercanos a 1 indican una distribución más uniforme entre clases, mientras que valores bajos reflejan el predominio de una o pocas clases.

Esto se debe a que la información de un suceso puede interpretarse como qué tan “sorprendente” es su ocurrencia: eventos muy probables aportan poca información, mientras que eventos raros aportan más. En este contexto, una mayor

entropía implica una distribución más uniforme y, por tanto, menor información promedio por evento individual, pero mayor equilibrio global entre clases. En resumen, una mayor entropía se asocia a un mayor balance de clases [Brownlee, 2020].

Esta medida fue clave para justificar la elección de GDS_R2 como caso principal de estudio.

En el preprocesamiento se emplearon dos estrategias. La primera fue un **filtro por correlación de Pearson**, que elimina atributos altamente correlacionados y reduce redundancia entre variables. La segunda fue **PCA**, aplicado tras estandarización, con el objetivo de reducir dimensionalidad conservando la mayor varianza posible en un nuevo espacio ortogonal de componentes.

Los clasificadores base utilizados fueron **Gaussian Naive Bayes**, **Random Forest**, **Regresión Logística** y **SVM**. Esta selección aporta diversidad de sesgos inductivos: un enfoque probabilístico, uno lineal, uno basado en márgenes y uno de árboles. Una de las ventajas de construir modelos de ensamble es que debido a que son modelos distintos, probablemente cometen errores distintos. Lo cual permite reducir la varianza de los modelos, evitando problemas como el sobre-ajuste [Bhuva, 2025].

En cuanto a los ensambles, se consideraron tres paradigmas. **Bagging** busca reducir varianza entrenando múltiples réplicas de un mismo estimador sobre subconjuntos re-muestreados. **Boosting** corrige errores de forma secuencial para reducir sesgo, aunque puede ser sensible al ruido y a una mala elección del clasificador base. **Stacking**, por su parte, combina varios modelos y entrena un meta-modelo sobre sus salidas, aprendiendo una regla de combinación más flexible.

Para la evaluación se utilizaron métricas macro-promediadas como *precision*, *recall* y *F1-score*, además de *accuracy*. El promedio macro (macro-average) es relevante en presencia de desbalance, debido a que tal como se enseña en la siguiente función:

$$\text{Macro Average} = \frac{1}{n} \sum_{i=1}^n x_i$$

Le asigna el mismo peso a cada clase. Considerando de la misma forma a clases con menor representación, sin sesgarse con aquellas que dominan el dataset [GeeksforGeeks, 2023]. Complementariamente, se emplearon curvas **ROC multiclase** mediante un esquema *One-vs-Rest* y el valor de **macro-AUC** como medida de comparativa.

Finalmente, el uso de **Leave-One-Out** dentro de `RandomizedSearchCV` se justificó por el tamaño del conjunto de entrenamiento. Esta estrategia permite entrenar en casi todas las observaciones en cada iteración y obtener una estimación mas realista del error pero asumiendo un

mayor costo computacional.

III. METODOLOGÍA

Organización general del pipeline: El experimento fue implementado de forma modular. El archivo `main.py` orquesta la ejecución en cuatro secciones: análisis inicial y generación del CSV procesado, carga y partición de datos, entrenamiento de modelos base y construcción/evaluación de ensambles. Esta organización permite comentar y descomentar bloques costosos según la etapa del experimento y favorece la reproducibilidad. A continuación se explica en mayor detalle cada sección del código.

III-A. Carga de datos y preparación inicial

Los datos crudos provienen del archivo 15 atributos `R0-R5.sav`. Tras la carga, se ejecutaron rutinas de EDA (Exploratory Data Analysis) para revisar valores faltantes, outliers y distribución de clases. Luego, los datos se exportaron a `data/processed/data.csv`, que pasa a ser la fuente principal para las siguientes etapas. Desde la columna GDS en adelante se identifican las variables objetivo, mientras que las columnas anteriores corresponden a atributos predictivos.

III-B. Selección de etiquetas

Debido al desbalance y el limitado numero de instancias presentes en el dataset, los investigadores responsables de su confección facilitaron dentro de la metadata 6 configuraciones de GDS (Geriatric Depression Scale) donde se construyen agrupaciones de clases para compensar dicho desbalance. A continuación se puede observar en la tabla I las diferentes GDS.

Codificación	Agrupación de categorías GDS	Nº de Clases
GDS	1, 2, 3, 4, 5, 6-7	6
GDS_1	1-3, 4-5, 6-7	3
GDS_2	1-2, 3, 4-7	3
GDS_3	1-3, 4-7	2
GDS_4	1, 2-4, 5-7	3
GDS_5	1, 2-3, 4-7	3

Tabla I: Agrupaciones de categorías GDS utilizadas para entrenamiento y prueba.

Durante la experimentación se incluyó GDS como referencia de control y se realizó un análisis de frecuencia de clases, cuyas distribuciones fueron evaluadas mediante su nivel de entropía con el fin de seleccionar otra configuración de GDS y comparar el desempeño de los modelos.

III-C. Partición de entrenamiento y prueba

Para ambas tareas se utilizó `train_test_split` con una división 60/40, `random_state=42` y estratificación. De esta manera, el conjunto de prueba conserva la proporción de clases observada en el dataset completo y se evita introducir sesgos adicionales por particiones no balanceadas.

III-D. Preprocesamiento

En `preprocessing.py` se definieron dos pipelines. El primero utiliza el transformador `CorrelationDropper`, una clase confeccionada por el equipo encargada de eliminar columnas con correlación absoluta superior a un umbral distribuido uniforme entre 0.6 y 0.95. Para su desarrollo se heredo de las clases `BaseEstimator` y `TransformerMixin` nativas de `sklearn` para poder implementarlas sin problemas con el resto del flujo de trabajo.

El segundo aplica `StandardScaler` seguido de PCA, optimizando el número de componentes con una distribución uniforme entre 7 y 15.

El propósito fue comparar una estrategia de selección de atributos con una de reducción de dimensionalidad, buscando en ambos casos favorecer el desempeño de *Naive Bayes* [Kavlakoglu, 2023]. Esto se debe a que dicho modelo asume independencia condicional entre las variables predictoras dado la clase.

A partir del teorema de Bayes:

$$P(C_k | X) = \frac{P(X | C_k) P(C_k)}{P(X)}$$

y bajo el supuesto de independencia (Que se busca asegurar con PCA y Pearson):

$$P(x_1, x_2, \dots, x_n | C_k) = \prod_{i=1}^n P(x_i | C_k)$$

se obtiene la formulación utilizada para clasificación:

$$P(C_k | x_1, x_2, \dots, x_n) = \frac{P(C_k) \prod_{i=1}^n P(x_i | C_k)}{P(x_1, x_2, \dots, x_n)}$$

III-E. Entrenamiento de modelos base

En esta etapa se definió un diccionario de clasificadores base utilizando implementaciones de `scikit-learn`, específicamente: *Logistic Regression*, *Naive Bayes*, *Random Forest* y *Support Vector Machine Classifier* (SVC). En este caso, se empleó la versión de *Naive Bayes* provista por la librería, y no la implementación propia desarrollada en el proyecto.

Junto con los modelos, se construyó un diccionario de espacios de búsqueda de hiperparámetros, donde cada parámetro se definió mediante distribuciones de probabilidad.

Esto permite que `RandomizedSearchCV` explore de forma eficiente distintas configuraciones durante la ejecución.

La función `random_train` recorre dos estrategias de preprocesamiento (PCA y filtrado por correlación de Pearson) junto con cada uno de los clasificadores definidos. Para cada combinación, se construye un Pipeline completo y se ejecuta `RandomizedSearchCV` con 10 iteraciones, utilizando `accuracy` como scorer y *Leave-One-Out*. Este procedimiento permite seleccionar automáticamente los mejores hiperparámetros.

Como resultado, la función devuelve un *DataFrame* que contiene, para cada combinación, las métricas de desempeño (*accuracy*, *precision*, *recall* y *F1-score*, todas en su versión macro), junto con los mejores hiperparámetros encontrados. Adicionalmente, se incluye una serialización del modelo entrenado.

Finalmente, estos resultados son persistidos en archivos CSV, permitiendo reutilizar los mejores modelos mas adelante, sin la necesidad de ejecutar todo el código de nuevo.

III-F. Manejo del desbalance

El desbalance se abordó en varias capas del pipeline. En primer lugar, se compararon distintas codificaciones objetivo mediante entropía. En segundo lugar, se utilizó partición estratificada. En tercer lugar, se reportaron métricas macro-promediadas para no favorecer de manera artificial a la clase mayoritaria. Además, la SVM se configuró con `class_weight='balanced'` y el *Naive Bayes* propio incorporó el parámetro `fit_prior` para aproximar la probabilidad de cada clase según su frecuencia observada.

III-G. Construcción de ensambles

Los modelos de ensamble se construyen a partir de los mejores resultados almacenados previamente en archivos CSV. La decisión de almacenar los modelos permite evitar la re-ejecución del proceso de `RandomizedSearchCV`, el cual es computacionalmente costoso debido al uso de *Leave-One-Out* y la exploración de múltiples hiperparámetros.

A partir de estos archivos, los modelos base son reconstruidos como flujos de trabajo completos (Pipelines), incluyendo su respectivo preprocesamiento y configuración óptima. En el caso de *Stacking*, todos los clasificadores recuperados se combinan mediante un `StackingClassifier`, utilizando Regresión Logística como meta-modelo.

Para *Bagging*, se selecciona el mejor modelo base según la métrica de *recall*. De forma similar, en *Boosting* se toma el mejor modelo según *accuracy*, se aplica su preprocesamiento correspondiente y se construye un `AdaBoostClassifier`.

Este enfoque permite explotar una de las principales ventajas de los ensambles: mejorar la capacidad de generalización y reducir la varianza, reduciendo así las posibilidades de un sobre-ajuste, lo cual considerando el desbalance del dataset, era bastante probable. En total, se construyen seis modelos de ensamble: tres asociados a la codificación GDS y tres correspondientes a la configuración de clases seleccionada en función de su entropía.

Esta etapa constituye el núcleo del laboratorio, ya que todo el proceso previo fue diseñado para identificar y reutilizar configuraciones óptimas con el fin de construir predictores más robustos.

III-H. Naive Bayes propio

Como etapa final de la metodología, se implementó un clasificador *Naive Bayes* desde cero con el objetivo de aplicar explícitamente el teorema de Bayes junto con el supuesto de independencia entre atributos, previamente descrito.

El modelo fue desarrollado en `naive_bayes.py` y hereda de `ClassifierMixin` y `BaseEstimator`, lo que permite integrarlo completamente al ecosistema de `scikit-learn`. Gracias a esto, el clasificador puede ser utilizado dentro de `Pipeline`, optimizado mediante `RandomizedSearchCV` y evaluado con métodos estándar como `fit`, `predict` y `predict_proba`, este último necesario para la construcción de curvas ROC en la etapa de resultados.

La implementación incorpora dos hiperparámetros principales: `alpha`, que aplica suavizado de Laplace para evitar probabilidades nulas (muy probables considerando un dataset desbalanceado), y `fit_prior`, que permite estimar las probabilidades a priori de cada clase a partir de los datos. Estas probabilidades representan el conocimiento previo sobre la distribución de las clases antes de observar las variables predictoras, lo que resulta especialmente relevante en escenarios desbalanceados, ya que permite al modelo considerar la frecuencia de cada clase [Kavlakoglu, 2023].

El objetivo de esta implementación es construir un modelo competitivo desde cero e integrarlo al flujo de trabajo de `scikit-learn`, manteniendo compatibilidad con los demás modelos para poder compararlos directamente.

IV. RESULTADOS

A continuación se presentan los resultados obtenidos, organizados en distintas subsecciones.

IV-A.

En primer lugar, se analiza la distribución de clases para cada configuración de la etiqueta, mostrando tanto la frecuencia absoluta como la entropía asociada. Este análisis permite evaluar el nivel de balance entre clases y fundamentar la

elección de los GDS utilizados.

Etiqueta	Distribución de clases	Entropía normalizada
GDS	149 / 500 / 298 / 108 / 42 / 20 / 2	0.7261
GDS_R1	947 / 150 / 22	0.4441
GDS_R2	649 / 298 / 172	0.8703
GDS_R3	947 / 172	0.6190
GDS_R4	149 / 906 / 64	0.5489
GDS_R5	149 / 798 / 172	0.7258

Tabla II: Distribución por codificación objetivo y entropía normalizada.

La Tabla II indica que GDS_R2 es la versión más balanceada del problema, por lo que será la seleccionada en conjunto a GDS para efectuar los experimentos. Además continuación, la grafica 1 de la entropía para cada codificación.

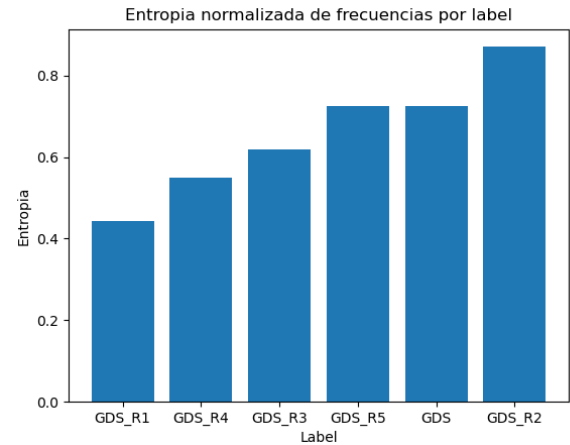


Figura 1: Entropía por cada configuración de etiqueta.

IV-B. Clasificadores base

La Tabla III nos presenta los resultados de los clasificadores base. Esto permite establecer una línea base de comparación antes de aplicar técnicas de ensamble. Cabe recordar que se entrenaron los mismos modelos base para GDS y GDS_R2

Etiqueta	Preprocesamiento	Clasificador	Recall
GDS	Pearson	Naive Bayes	0.159
GDS	Pearson	Random Forest	0.141
GDS	Pearson	Logistic Regression	0.141
GDS	Pearson	SVM	0.146
GDS	PCA	Naive Bayes	0.135
GDS	PCA	Random Forest	0.142
GDS	PCA	Logistic Regression	0.146
GDS	PCA	SVM	0.158
GDS_R2	Pearson	Naive Bayes	0.599
GDS_R2	Pearson	Random Forest	0.632
GDS_R2	Pearson	Logistic Regression	0.619
GDS_R2	Pearson	SVM	0.645
GDS_R2	PCA	Naive Bayes	0.666
GDS_R2	PCA	Random Forest	0.628
GDS_R2	PCA	Logistic Regression	0.619
GDS_R2	PCA	SVM	0.645

Tabla III: Recall de los clasificadores base para ambas codificaciones.

Se observa una diferencia clara entre ambas codificaciones. Los modelos entrenados con GDS presentan valores de *recall* muy bajos, evidenciando que no logran minimizar los falsos negativos. En cambio, al utilizar GDS_R2, todos los modelos mejoran este indicador, mostrando una mayor capacidad para detectar casos positivos, aunque aún insuficiente para un uso en producción.

Se utiliza *recall* como métrica principal debido al contexto médico del problema, donde es más crítico evitar falsos negativos que falsos positivos; es decir, es preferible sobre-diagnosticar que pasar por alto una condición existente.

IV-C. Modelos de ensemble y Own Naive Bayes

Luego, se presentan los modelos de ensemble junto con sus respectivas curvas ROC. Para la codificación GDS, se incluye únicamente el modelo de *Stacking*, mientras que para GDS_R2 se analizan los tres enfoques: *Stacking*, *Bagging* y *Boosting*. Las matrices de confusión correspondientes se incluyen en los anexos.

Finalmente, se incorpora el desempeño del modelo *Naive Bayes* implementado desde cero, junto con su curva ROC. Para complementar el análisis, se presenta una tabla resumen con el área bajo la curva (AUC) de todos los modelos considerados, permitiendo una comparación directa de su rendimiento.

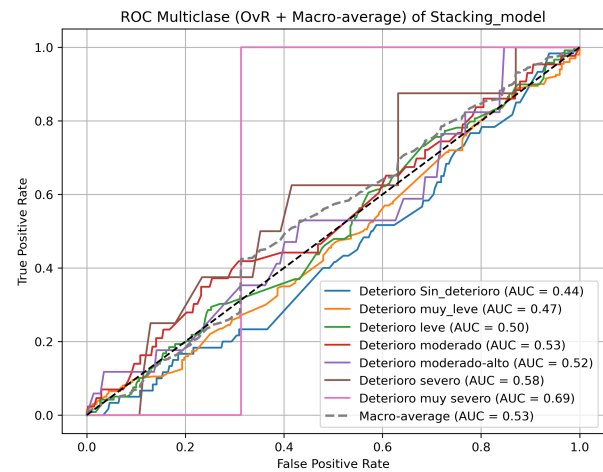


Figura 2: Curva ROC multicase para Stacking en GDS.

En el grafico 2, un grafico OvR (One Vs Rest) donde cada una de las clases se compara con el resto, se puede observar como es que el promedio de las clases se comporta tan solo un tanto mejor que el azar. Tambien es interesante mencionar que las clases a medida que poseen menor frecuencia en el dataset (deterioro severo o muy severo) poseen una curva mucho menos suave.

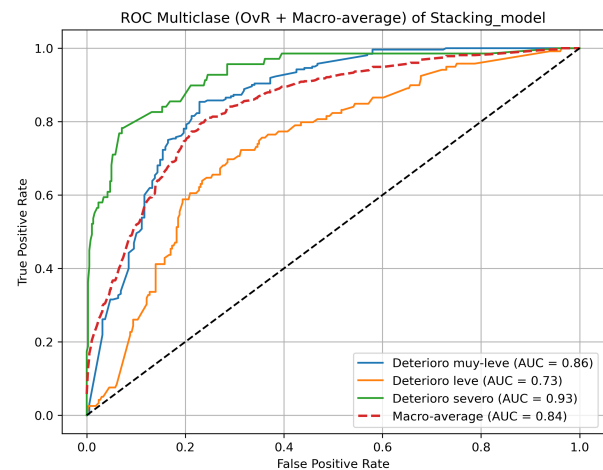


Figura 3: Curva ROC multicase para Stacking en GDS_R2.

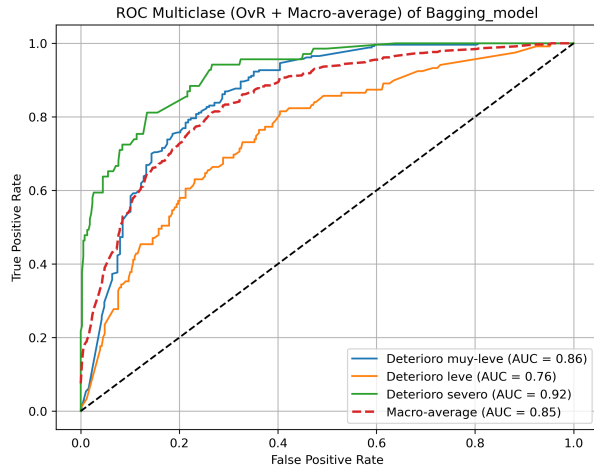


Figura 4: Curva ROC multiclase para Bagging en GDS_R2.

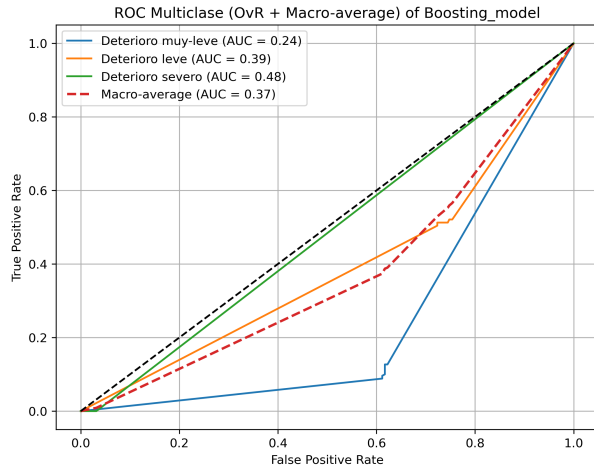


Figura 5: Curva ROC multiclase para Boosting en GDS_R2.

De los graficos 3, 4 y 5 que presentan el desempeño de los modelos de ensemble en GDS_R2 se puede extraer información variada. Primero la notoria diferencia con respecto al desempeño de Stacking en GDS. También el mejor rendimiento, si nos enfocamos solo en las clases, lo obtuvo la clase de deterioro severo en Stacking, con un area bajo la curva de 0.93, siguiendole la clase de deterioro severo en Bagging con un AUC de 0.92.

Modelo de Ensemble	Macro AUC
Stacking	0.84
Bagging	0.85
Boosting	0.37

Tabla IV: Desempeño de modelos de ensemble (macro AUC) para GDS_R2.

Si nos enfocamos en los Macro Average, podemos observar como es que el mejor rendimiento medio lo obtuvo Bagging con un rendimiento bastante competitivo por parte de Stacking, mientras que Boosting no presento resultados

destacables.

Para finalizar se presenta la gráfica ?? correspondiente a la curva ROC y el area bajo la curva asociada al desempeño del modelo Naive Bayes diseñado por el equipo.

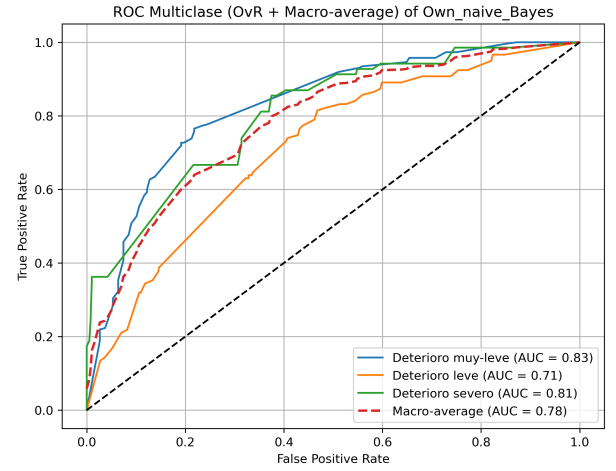


Figura 6: Curva ROC multiclase para Naive Bayes Propio en GDS_R2.

Considerando que es un modelo independiente, presento resultados bastante competitivos con respecto a los modelos de ensemble.

En el anexo se podrán encontrar todas las matrices de confusión asociadas a los modelos de ensemble y naive bayes de GDS_R2.

V. DISCUSIÓN

A partir de los resultados obtenidos, se puede observar que el factor más determinante en el desempeño de los modelos no fue necesariamente el algoritmo utilizado, sino la forma en que se definió la variable objetivo. La diferencia entre GDS y GDS_R2 es clara: mientras que en el primer caso los modelos no logran capturar correctamente los patrones (presentando un *recall* muy bajo), en el segundo caso se aprecia una mejora consistente en todos los clasificadores. Esto refuerza la importancia del balance de clases en problemas multiclase, especialmente en contextos con datos limitados.

En cuanto a los clasificadores base, ninguno logró por sí solo un desempeño suficientemente robusto. Sin embargo, sí fueron capaces de capturar cierta estructura en los datos cuando se utilizó GDS_R2. Esto valida el enfoque de utilizarlos como componentes dentro de modelos de ensemble, en lugar de depender de ellos de forma individual.

Respecto a los ensambles, se observa que tanto *Bagging* como *Stacking* logran mejoras claras en términos de macro-AUC, alcanzando valores de 0.85 y 0.84 respectivamente. Esto es consistente con la teoría, ya que ambos métodos buscan

reducir la varianza combinando múltiples modelos. En contraste, *Boosting* presenta un desempeño considerablemente inferior (0.37), lo que sugiere que este enfoque no se adapta bien al problema, probablemente debido a la sensibilidad al ruido y al desbalance presente en los datos.

También es interesante notar que el modelo de *Naive Bayes* implementado desde cero logró resultados competitivos en comparación con los ensambles. Esto indica que, a pesar de su simplicidad y de la fuerte suposición de independencia, el modelo es capaz de capturar relaciones relevantes en los datos cuando se integra correctamente dentro del pipeline. Parte de este buen desempeño también puede atribuirse a los hiperparámetros definidos, particularmente `fit_prior`, que permite incorporar el desbalance de clases, y `alpha`, que aplica suavizado de Laplace y evita problemas asociados a probabilidades nulas.

En general, los resultados muestran que la combinación de una buena representación del problema (en este caso, una codificación más balanceada) junto con técnicas de ensamble permite obtener mejoras significativas en el desempeño. Sin embargo, los valores alcanzados aún sugieren que existe espacio para mejorar antes de considerar una aplicación en un contexto real.

VI. CONCLUSIONES

En este trabajo se logró construir un pipeline completo y reproducible para abordar un problema de clasificación multiclase en un contexto de deterioro cognitivo. Los resultados evidencian que la elección del GDS es un factor crítico, siendo la configuración más balanceada la que permitió obtener mejoras significativas en el desempeño. Además, el uso de la entropía como medida del nivel de desbalance de clases resultó especialmente útil.

En relación a los objetivos planteados, estos fueron cumplidos de manera satisfactoria. Se logró analizar el balance de las distintas codificaciones y seleccionar una representación adecuada; se compararon estrategias de preprocesamiento como correlación de Pearson y PCA,¹ ambas demostrando ser útiles en la reducción de redundancia² y dimensionalidad; se entrenaron clasificadores base mediante³ `RandomizedSearchCV` con validación *Leave-One-Out*; se⁴ construyeron y evaluaron modelos de ensamble; y finalmente, se implementó correctamente un modelo *Naive Bayes* propio en el flujo de trabajo.

Respecto al desempeño, se observó que los clasificadores base, aun cuando individualmente presentan resultados limitados, logran aportar valor cuando son ensamblados. En particular, *Bagging* y *Stacking* demostraron que la combinación de modelos puede mejorar la generalización, validando la idea de que modelos débiles pueden convertirse en predictores² más robustos al trabajar en conjunto.³

La validación mediante *Leave-One-Out* resultó adecuada dado el tamaño del dataset, permitiendo aprovechar al máximo la información disponible sin requerir un costo computacional alto. Además, contribuyó a obtener estimaciones más estables en un escenario con clases desbalanceadas.

Por otra parte, la implementación propia de *Naive Bayes* mostró un desempeño competitivo. Este resultado también sugiere que modelos simples pueden ser efectivos bajo una adecuada configuración.

Como trabajo futuro, se propone explorar otras estrategias para el tratamiento del desbalance de clases, más allá del uso de codificaciones definidas, tal vez técnicas de re-muestreo o generación sintética de datos. Asimismo, resultaría interesante una metodología que incluya *Bagging* utilizando el *Naive Bayes* propio como modelo base, considerando que ambos, por separado, mostraron un buen desempeño. Esto podría potenciar aún más los resultados al combinar un modelo simple y efectivo dentro de un ensamble.

Finalmente, se sugiere continuar explorando nuevas configuraciones y enfoques que permitan mejorar la detección de clases minoritarias, manteniendo como prioridad la reducción de falsos negativos en este tipo de aplicaciones.

VII. CÓDIGO Y REPRODUCIBILIDAD

El proyecto está organizado de manera modular. `main.py` actúa como orquestador, `eda.py` concentra utilidades exploratorias, `preprocessing.py` define transformadores y pipelines, `classifiers.py` contiene configuraciones de clasificadores base, `train.py` implementa la búsqueda de hiperparámetros, y los archivos `stacking.py`, `bagging.py` y `boosting.py` encapsulan cada ensamble.

Una buena práctica importante fue centralizar la configuración en `config.py`. Allí se definen rutas, nombres de clases, un Enum para los modelos y un diccionario `MODELS` que evita depender de *magic strings*. Esto mejora mantenibilidad y disminuye errores de integración.

```
class Model_names(Enum):  
    STACKING_NAME = "Stacking_model"  
    BOOSTING_NAME = "Boosting_model"  
    BAGGING_NAME = "Bagging_model"  
    OWN_NAIVE_NAYES = "Own_naive_Bayes"
```

Código 1: Enumeración centralizada de modelos en `config.py`

El nuevo *Naive Bayes* propio siguió la interfaz estándar de `scikit-learn`, lo que permitió integrarlo directamente a pipelines y búsquedas aleatorias:

```
class NaiveBayesClassifier(ClassifierMixin,  
    BaseEstimator):  
    def __init__(self, alpha=1.0, fit_prior=  
        True):  
        self.alpha = alpha  
        self.fit_prior = fit_prior
```


Código 2: Definición base del clasificador propio

Otra buena práctica fue estructurar `main.py` por secciones comentadas para evitar relanzar experimentos pesados cuando ya existen CSV e imágenes generadas. Esto hace el flujo más reproducible y más eficiente en tiempo de desarrollo. En el repositorio actual existen resultados exportados e imágenes generadas, por lo que el análisis es replicable siempre que se utilice el entorno conda definido en `environment.yml`.

APÉNDICE A ANEXOS

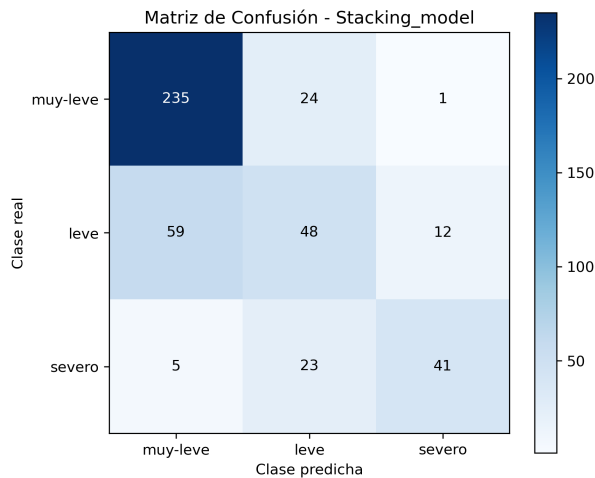


Figura 7: CM multiclase para Stacking en GDS_R2.

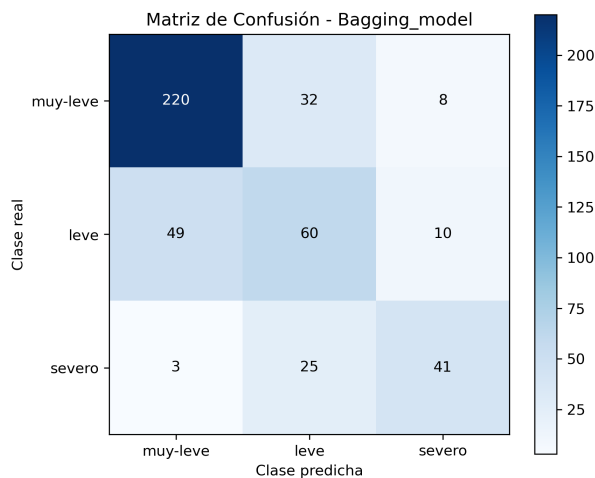


Figura 8: CM multiclase para Bagging en GDS_R2.

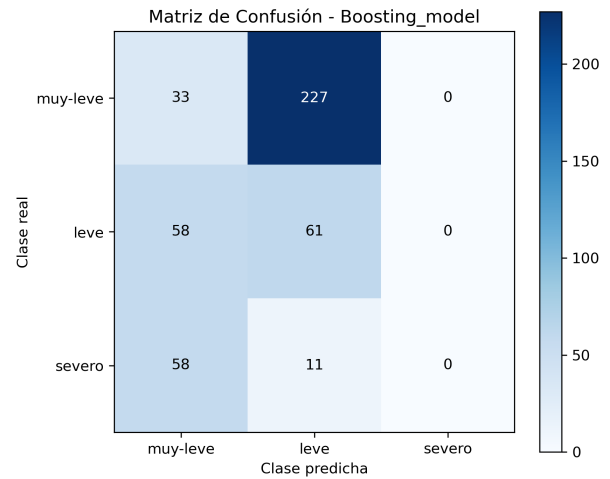


Figura 9: CM multiclase para Boosting en GDS_R2.

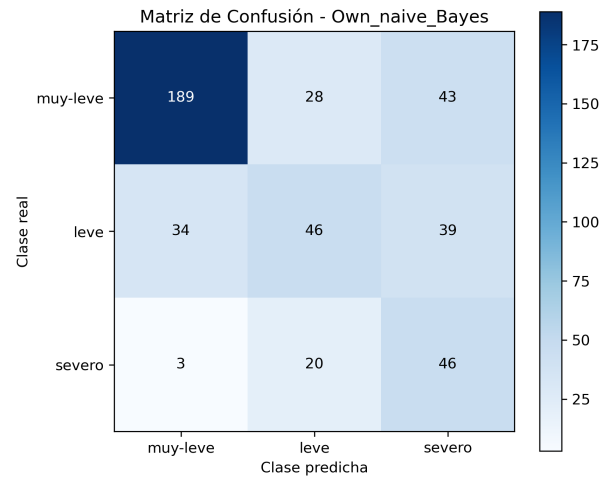


Figura 10: CM multiclase para Own Naive Bayes en GDS_R2.

APÉNDICE B RÚBRICA DE AUTOEVALUACIÓN

Criterio	¿Cumplido?
Introducción clara	Sí
Marco teórico adecuado	Sí
Metodología detallada	Sí
Resultados completos y visuales	Sí
Discusión crítica	Sí
Conclusiones reflexivas	Sí
Calidad y estructura del código	Sí
Presentación clara y sin errores	Sí
Reproducibilidad	Sí
Aportes adicionales	Sí

Tabla V: Tabla de autoevaluación.

REFERENCIAS

- [Bhuva, 2025] Bhuva, L. (2025). Understanding ensemble learning: A comprehensive guide. Medium, Accessed: 2026-04-23.
- [Brownlee, 2020] Brownlee, J. (2020). A gentle introduction to information entropy. Accessed: 2026-04-23.

[Fundación Pasqual Maragall, 2023] Fundación Pasqual Maragall (2023). Escala fast. Accessed: 2026-04-23.

[GeeksforGeeks, 2023] GeeksforGeeks (2023). Macro average vs weighted average in machine learning. Accessed: 2026-04-23.

[Kavlakoglu, 2023] Kavlakoglu, E. (2023). What are naïve bayes classifiers? IBM, Accessed: 2026-04-23.